DATA ENGINEERING PROJECT — PROPOSAL & TECHNICAL PLAN

Mutual Fund Analytics Platform

Turning a daily price email into historical tracking, performance analytics, and alerts

Working title: "FundLens" — a personal fund-tracking data platform

Data source: Daily fund price disclosure (NAV sheet) emailed by an asset management company

Author: _____ (CS Graduate)

Version: 1.0 • Draft for review

Document type: Standalone proposal (stakeholder brief + business proposal + technical report)

About this document

This is a single, self-contained document written in a plain, README style. It has three parts and needs no other paper to be understood:

1. **Part 1 — Stakeholder Brief:** the problem in the stakeholder's own words, and what they want at the end.
2. **Part 2 — Business Proposal:** what will be built, what it looks like, and why it is worth doing.
3. **Part 3 — Technical Report:** how it will be built — architecture, tools (with trade-offs), the ingest→model→transform→serve pipeline, an agile plan, and risks.

> **IMPORTANT DISCLAIMER**
> This is an **informational and educational** data project. It reports and analyses publicly disclosed prices. It is **not** financial advice and does not tell anyone to buy or sell. Any "signals" are descriptive notifications (e.g. "this fund moved unusually today"), not recommendations.

Mini-glossary (so this reads standalone)

Term	Plain meaning
NAV	Net Asset Value — the per-unit price of a mutual fund on a given day.
ETL / ELT	Extract, Transform, Load — the steps to pull data, clean it, and store it for use.
Pipeline	The automated chain that moves data from source to dashboard every day.
Medallion (Raw / Clean / Curated)	A common way to layer a data platform: keep the untouched source, then a cleaned version, then business-ready tables.
SCD Type 2	"Slowly Changing Dimension" — a modelling trick to keep history when a fund's details (name/category) change.
dbt	A popular tool to write, test, and document SQL transformations.

Part 1 — Stakeholder Brief

Who is the stakeholder?

The stakeholder is a **retail investor** who holds units in several mutual funds from one asset-management company. Every business day, the company emails them a single PDF sheet listing that day's price (NAV) for every fund. The stakeholder is not a finance professional and is not a programmer — they just want to keep sensible track of their money without spending time on it.

The problem, in their words

> **STAKEHOLDER STATEMENT**
> "Every day I get one PDF with today's prices for all the funds. That's it — just today. I have no history, so I can't see how a fund has done over the last month or year. I can't easily compare funds against each other. To keep any record I'd have to open the email every single day and copy numbers into a spreadsheet by hand, which I never keep up with. I want to know which funds are doing well over time, understand the ups and downs, and get a nudge when something notable happens — without reading a PDF every morning."

Why this is a real problem

- **No memory.** The source is a daily snapshot. Yesterday's sheet is gone unless someone saved it. There is no built-in history to analyse.
- **Manual and error-prone.** Building history by hand means opening ~250 emails a year and copying numbers — nobody does this reliably.
- **Hard to compare.** A raw price tells you little. "Is this fund good?" needs returns over time, and comparison against other funds — which the PDF cannot give.
- **No timely awareness.** A notable move (a fund jumping or dropping unusually) is invisible until you happen to read that day's sheet.

What the stakeholder wants at the end

#	Want	What "done" looks like
1	Automatic history	Every daily sheet is captured and stored forever, with no manual work.
2	Price-over-time view	A chart of any fund's NAV over any period (1M / 3M / 1Y / all).
3	Performance analytics	Rolling returns, volatility, drawdown, and a risk-adjusted score per fund.
4	A leaderboard	"Best fund over the last N months," with the window and category selectable.
5	Alerts	A message (e.g. Telegram/email) when a fund moves unusually, or the daily sheet fails to arrive.
6	Low effort & trust	It runs itself daily; the numbers are validated so they can be trusted.

What success looks like (acceptance criteria)

- The platform ingests each daily sheet automatically, with **zero manual steps** on a normal day.
- It holds a continuous, gap-free history for all funds (from project start, plus any backfilled sheets).
- The dashboard answers "how has fund X done?" and "which fund is best over the last N months?" in a few clicks.
- If the daily email is missing or the numbers look wrong, the stakeholder is **alerted rather than shown bad data**.

Out of scope (kept honest and simple)

No buy/sell recommendations, no portfolio-value or tax tracking, no prediction of future prices. The project is about **reliable history + clear analytics + notifications**, nothing more.

Part 2 — Business Proposal

The idea in one paragraph

Build a small, automated **data platform** that reads the daily fund-price email by itself, keeps a permanent and validated history of every fund's NAV, calculates useful performance measures, and presents everything in a simple dashboard — plus sends a short alert when something is worth knowing. The stakeholder stops reading PDFs and instead gets clean history, clear comparisons, and timely nudges.

What the end product looks like

1. A dashboard (the main deliverable)

- **Fund explorer:** pick a fund, see its price line over any time range, with returns and risk numbers beside it.
- **Leaderboard:** a sortable table of funds ranked by return over a chosen window (1M / 3M / 6M / 1Y / YTD), filterable by fund category (e.g. equity vs income).
- **Compare view:** two or more funds on the same chart, rebased to 100 so they can be compared fairly.
- **Data-health strip:** "last updated" date and a green/red indicator so the stakeholder trusts what they see.

2. Alerts (the "don't-make-me-check" deliverable)

- **Freshness alert:** "Today's price sheet did not arrive / could not be read" — so silence never hides a failure.
- **Movement alert:** "Fund X moved +/- more than its usual daily range today" — a factual notification, not advice.

> **USER JOURNEY (BEFORE → AFTER)**
> **Before:** open email → read one PDF → forget → no history → guess which fund is doing well.
> **After:** ignore the email → open dashboard when curious → get alerts when something matters → decisions backed by real history.

Who benefits, and how

Beneficiary	Benefit
The investor (primary)	Saves daily effort; sees real trends and fair comparisons; is nudged only when it matters.
Anyone with the same funds	The same dashboard works for family/friends holding the same funds — it is not personalised to one portfolio.
The builder (you)	A genuinely used, end-to-end data platform on real, messy data — a strong portfolio piece (see resume note below).

Why this is worth building

- **Real and used daily.** It solves a problem the builder actually has, on data that arrives every day — not a one-off demo.
- **Commercially proven pattern.** Global tools (e.g. portfolio/price trackers) do exactly this for stocks; they simply don't cover these local funds or this quirky email-PDF source. This project fills that gap.
- **Cheap to run.** The data is tiny (a few numbers per day), so it can run for near-zero cost on a laptop or a small always-on machine.

Assumptions & constraints

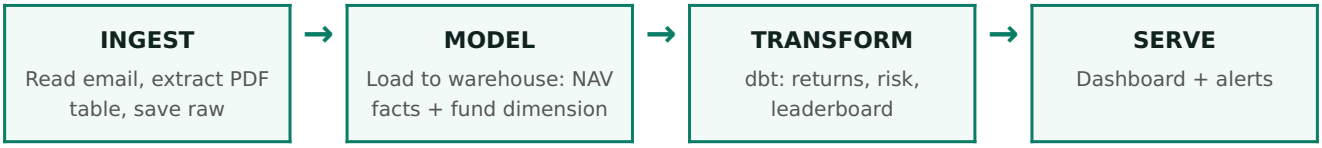
- The daily email keeps arriving and its PDF layout stays roughly stable (the design must survive small format changes and shout when it can't parse).
- Historical sheets before project start may be limited; history is built forward from day one, with backfill of any old emails that can be found.
- This is a personal-scale project; it does not need enterprise infrastructure and must not pretend to.

> **RESUME FRAMING (HONEST)**
> On a CV this reads as: "*Designed and built an automated, tested daily ETL pipeline over a brittle real-world source (email + PDF), with historical modelling, dbt-tested transformations, orchestration with failure alerting, and a BI dashboard.*" That sentence signals production concerns — which is what actually impresses reviewers, more than any single tool name.

Part 3 — Technical Report

Architecture at a glance

The platform follows the standard **medallion** layering (Raw → Clean → Curated) with a serving layer on top. Everything is driven by one daily scheduled job.



> **LAYER MEANING**
> **Raw zone** = the original PDF + the exact parsed numbers, immutable, partitioned by date. Never edited — it lets us re-run everything if logic changes.
> **Clean (warehouse staging)** = typed, de-duplicated, one row per fund per day.
> **Curated (marts)** = business tables the dashboard reads: returns, risk metrics, leaderboard.

Stage 1 — Ingest

What happens

1. Connect to the mailbox over **IMAP** and find today's price email (by sender + subject + date).
2. Save the attached PDF **unchanged** into the raw zone as `raw/nav/YYYY-MM-DD.pdf`.
3. Extract the price table from the PDF and write the parsed rows as `raw/nav/YYYY-MM-DD.parquet` (keep both original + parsed).
4. Run first-pass checks: did we get a table? Is the fund count in the expected range? Are prices positive numbers?

Tools & why

Job	Chosen	Why this, not the runner-up
Get the email	Python `imaplib`	Standard library, no cost, works with normal mailboxes. *Runner-up:* a paid inbox API — unnecessary cost/complexity for one mailbox.
Parse the PDF table	`pdfplumber`	Pure-Python, easy install, good at line-based tables. *Runner-up:* `camelot` is powerful but needs Ghostscript/system deps — heavier for a simple, consistent sheet. (Keep camelot as a fallback if layout gets tricky.)
Raw storage format	Parquet files + original PDF	Columnar, typed, tiny, re-readable by everything. *Runner-up:* CSV — loses types and is bulkier; fine as a debug export only.

Stage 2 — Model (load & data model)

The data model

- `fct_nav` **(fact):** one row per `(date, fund_id)` → NAV, offer/redemption price if present, category snapshot.
- `dim_fund` **(dimension, SCD Type 2):** fund id, name, category, with `valid_from` / `valid_to` so that if a fund is renamed or recategorised, history stays correct.
- `dim_rate` **(optional):** a risk-free rate (e.g. T-bill / policy rate) needed for a Sharpe-style score — a small second source that adds depth.

> **WHY SCD TYPE 2 MATTERS HERE**
> It is a small choice that signals real data-modelling understanding: instead of overwriting a fund's category when it changes, we keep both versions with dates — so a 1-year return never mixes two different definitions of the same fund.

Where it lives

Job	Chosen	Why this, not the runner-up
Warehouse	**PostgreSQL** (in Docker)	Free, ubiquitous, SQL-standard, plays perfectly with dbt and every BI tool — and "Postgres + dbt" is a highly recognised combo on job posts. *Runner-up:* **DuckDB** is excellent and even simpler for single-user analytics, but Postgres better demonstrates a real client/server warehouse and multi-tool serving. (DuckDB is a great local alternative and is noted as such.)
Load method	Small Python loader (idempotent upsert by date)	Re-running a day must not create duplicates. *Runner-up:* a heavy ingestion framework (Airbyte/Spark) — massive overkill for kilobytes/day.

Stage 3 — Transform

What gets computed (all in dbt)

- **Returns:** daily return, and rolling 1M / 3M / 6M / 1Y / YTD returns per fund.
- **Risk:** annualised volatility (std-dev of daily returns) and maximum drawdown.
- **Risk-adjusted score:** a Sharpe-style ratio using the risk-free rate from `dim_rate`.
- **Leaderboard mart:** funds ranked by return over each window, within category.
- **Movement flag:** today's move vs the fund's own typical daily range (for the alert).

Why dbt (and why not the alternatives)

Option	Verdict
dbt (chosen)	SQL transforms that are **version-controlled, tested, documented, and lineage-aware**. It is the current industry-standard transform layer and a strong resume keyword. Perfect fit for this kind of tabular metric-building.
Plain SQL scripts	Works, but no built-in tests, no lineage, no docs, easy to let rot. dbt gives all of that almost for free.
pandas in Python	Fine for prototyping, but logic hides in code, is harder to test as data, and doesn't push work into the warehouse. Kept only for quick exploration.
Spark	Built for big data. Here it would be like renting a truck to carry a envelope — pure over-engineering.

Data-quality tests (run every day)

- NAV is present and **> 0**; no nulls in keys.
- `(date, fund_id)` is **unique** (no duplicate loads).
- No **missing business days** in the series (gaps get flagged).
- Fund count per day is within an expected range (catches partial parses).
- Day-over-day move is within a sane bound (catches a decimal-point / parsing error).

Tests are written as dbt tests (plus a few custom ones). If a test fails, the pipeline stops before bad numbers reach the dashboard, and an alert fires. *Great Expectations is a heavier alternative considered but not needed at this scale.*

Stage 4 — Serve

Job	Chosen	Why this, not the runner-up
Dashboard	**Metabase**	Fast to stand up, connects straight to Postgres, gives clean charts and filters with little code, looks professional. *Runner-up:* **Streamlit** — more custom/coded control, but more build effort for standard charts. (Streamlit is the better pick if heavy custom interactivity is needed later.)
Alerts	Telegram bot (or email) from a pipeline task	Free, instant, phone-friendly. *Runner-up:* email-only — works but easier to ignore; Telegram feels like a real push notification.

Orchestration & operations

Concern	Chosen	Why this, not the runner-up
Scheduler / orchestrator	**Apache Airflow**	The most widely-requested orchestrator on DE job posts; mature, with retries, scheduling, backfill, and failure callbacks built in. *Runner-up:* **Dagster** has a nicer local developer experience and an asset-based model — genuinely a strong choice; Airflow is chosen mainly for **resume recognisability** and its huge ecosystem. Either is defensible; the report picks one and stays consistent.
Packaging	**Docker Compose**	One command brings up Postgres + Airflow + Metabase; makes the project reproducible on any machine and demo-able to a reviewer. *Runner-up:* bare-metal installs — fragile and hard for anyone else to run.
CI	**GitHub Actions**	On each push, run `dbt build` against a test database and lint/format code — cheap polish that proves the pipeline is tested, not just written. *Runner-up:* no CI — misses an easy credibility win.

> **THE DAILY DAG, IN WORDS**
> **1)** fetch email → **2)** save raw PDF + parse to Parquet → **3)** load to Postgres (idempotent) → **4)** `dbt build` (models + tests) → **5)** refresh leaderboard/marts → **6)** evaluate movement flags & send alerts. Any failure → retry, then alert. Missing email by a cut-off time → freshness alert.

Final tech stack (kept deliberately lean)

Layer	Tool
Ingestion	Python (`imaplib` , `pdfplumber`) → Parquet raw zone
Warehouse	PostgreSQL (Docker) [alt: DuckDB]
Transform & tests	dbt
Orchestration	Apache Airflow [alt: Dagster]
Serving	Metabase dashboard + Telegram/email alerts [alt: Streamlit]
Ops	Docker Compose, GitHub Actions CI, Git

Agile plan — phases & sprints

Solo project, so "sprints" are ~1–1.5 week focused blocks. Each ends with something that works end-to-end, however small. Total: roughly **6–8 weeks** part-time.

> SPRINT 0 • ~3 DAYS
> **Setup & walking skeleton**
> Repo, Docker Compose (Postgres + Airflow), Git, README. Manually run one email through parse→load so a single day appears in Postgres. **Done:** one real day of data lands, by hand.

SPRINT 1 • ~1 WK

Automated ingest + raw zone

IMAP fetch, PDF parse with `pdfplumber`, raw PDF + Parquet saved, idempotent load, first sanity checks. Wire it into a daily Airflow DAG. **Done:** the pipeline captures each day by itself.

SPRINT 2 • ~1 WK

Data model + quality

Build `fct_nav` and `dim_fund` (SCD2) in dbt; add the core dbt tests (uniqueness, not-null, positivity, gaps). **Done:** trustworthy, tested clean layer.

SPRINT 3 • ~1–1.5 WK

Analytics marts

Rolling returns, volatility, drawdown, Sharpe-style score (add `dim_rate`), and the leaderboard mart. **Done:** the numbers the stakeholder actually wants exist as tables.

SPRINT 4 • ~1 WK

Serve: dashboard

Metabase: fund explorer, leaderboard, compare (rebased) view, data-health strip. **Done:** stakeholder can self-serve answers.

SPRINT 5 • ~4 DAYS

Alerts + hardening

Freshness alert, movement alert, DAG retries/failure callbacks, backfill script for old emails, GitHub Actions CI. **Done:** runs unattended and tells you when it can't.

SPRINT 6 • ~3 DAYS

Polish & write-up

README with architecture diagram, screenshots, a short "how it handles failure" section, and demo notes. **Done:** portfolio-ready.

Risks & how they're handled

Risk	Mitigation
PDF layout changes silently	Parse-count and schema checks; if they fail, stop and alert instead of loading garbage. Keep the raw PDF so a fixed parser can re-run history.
Email missing / late	Freshness alert after a cut-off time; DAG retries; backfill fills the gap when it arrives.
Wrong number slips in	Positivity + day-over-day bound tests catch decimal/parse errors before the dashboard.
Over-engineering	Deliberately lean stack; streaming/Spark explicitly rejected as inappropriate for tiny daily data.
Source terms of use	Data is used personally for analysis, not redistributed; note this in the README.

Future extensions (nice-to-have, not committed)

- Add a second AMC's disclosures to compare across companies.
- Optional portfolio overlay (units held → value over time) — kept out of MVP.
- Swap Metabase for a custom Streamlit app if richer interactivity is wanted.

End of document. This proposal is self-contained: Parts 1–3 above cover the problem, the product, and the full build plan with no external references required.